

Git - Cheat Sheet

Git is the version control system that powers nearly every modern software project. Linus Torvalds wrote it in 2005 to manage the Linux kernel after the project's previous tool (BitKeeper) became unworkable. The design choices that came from kernel-scale development (distributed, fast, branchable) are why it took over the world.

What Git is

Git is a distributed version control system. Every clone has the full history. There's no central server in the protocol sense. GitHub, GitLab, and Bitbucket are convenient hosts, but any clone can become the source of truth.

Three properties shape everything else:

- Content-addressable storage. Every file, tree, and commit is identified by the SHA-1 (now SHA-256) hash of its content.
- Stores snapshots. Each commit stores the entire tree state. Deltas are an internal storage detail.
- Cheap branches. Branches are 40-byte references. Creating one is instantaneous.

Core concepts

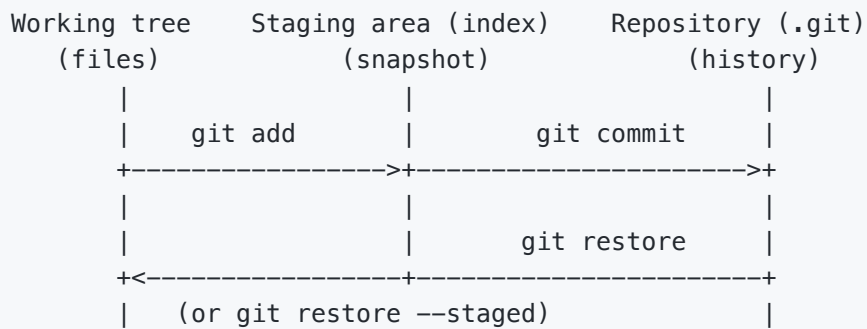
Git tracks four object types in `.git/objects/` :

Object	What it stores
Blob	File contents (no name, just bytes)
Tree	Directory structure: filenames -> blob/tree hashes + mode
Commit	Tree hash + parent commit(s) + author + message
Tag	Annotated tag with metadata

Plus:

- `HEAD` : a reference to the current commit (usually points to a branch).
- Branches: movable refs that point to a commit. As you commit, the branch moves forward.
- Refs: pointers in `.git/refs/` . Branches and tags are refs.
- Index (staging area): the next commit being built.

The three areas



The staging area is what makes Git different from older VCS. You build the next commit incrementally, choosing exactly what goes in.

Setup and first commit

`git config --list` shows everything currently configured.

```
# Configure once globally
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global pull.rebase false # or 'true' if you prefer rebasing on pull

# Start a new repo
git init
git add .
git commit -m "initial commit"

# Or clone an existing one
git clone https://github.com/org/repo.git
cd repo
```

Day-to-day commands

The most-used commands:

```

git status          # what's changed
git diff            # unstaged changes
git diff --staged   # staged changes
git diff main..feature # difference between branches

git add file.txt    # stage a file
git add -p          # interactively pick hunks
git add .           # stage everything in current dir

git commit -m "message"
git commit -a -m "msg" # stage modified tracked files and commit

git log             # commit history
git log --oneline   # compact view
git log --graph --all # ASCII commit graph
git log --author="alice"
git log -p file.txt # patches for a specific file

git show <commit> # what changed in a commit
git blame file.txt # who last touched each line

```

`git status` is the command you'll type most. When in doubt, type it.

Branching

```

git branch          # list local branches
git branch -a       # local + remote
git branch feature-x # create a branch
git checkout feature-x # switch (older syntax)
git switch feature-x # switch (Git 2.23+ syntax, cleaner)
git checkout -b new-branch # create + switch
git switch -c new-branch # same, modern
git branch -d old-branch # delete (only if merged)
git branch -D old-branch # force delete
git branch -m old-name new-name # rename current branch

```

`git switch` and `git restore` (Git 2.23+) replace the overloaded `git checkout`. Reach for them when you can.

Merging

```

git checkout main
git merge feature-x

```

Three behaviors depending on the topology:

- Fast-forward: if `main` hasn't moved since `feature-x` was created, `main` just points forward.

- Three-way merge: if both branches diverged, Git creates a merge commit combining them.
- Conflict: if the same lines changed in both, you fix them manually.

Force a merge commit even when fast-forward is possible:

```
git merge --no-ff feature-x
```

This preserves the branch history visibly. Common policy on shared `main`.

Rebasing

`rebase` rewrites commits onto a different base.

```
git checkout feature-x
git rebase main
```

This takes your commits and replays them on top of `main`'s tip. The result is a linear history.

Interactive rebase (the killer feature):

```
git rebase -i HEAD~5    # edit last 5 commits
```

You get an editor with options: `pick`, `reword`, `edit`, `squash`, `fixup`, `drop`. Use it to squash related commits, reword bad messages, reorder commits, or drop accidental commits.

Golden rule of rebase: don't rebase commits that are public. Rewriting history on a shared branch breaks everyone's clones.

Merge vs rebase

Aspect	Merge	Rebase
History shape	Branches visible	Linear
Commit hashes	Preserved	Rewritten
Safe on shared branches	Yes	No
Conflict resolution	Once per merge	Possibly once per commit
Use for	Integrating finished features	Cleaning up before merge

Common workflow: rebase your feature branch on `main` before opening a PR, then merge with `--no-ff`. You get a clean linear feature plus a visible merge point.

Remote repositories

```
git remote -v                # list remotes
git remote add origin <url>  # add a remote
git remote set-url origin <new-url>
git remote remove old-remote

git fetch                    # download refs without changing your work
git fetch --prune            # also delete remote-tracking refs that no longer exist
git pull                    # fetch + merge (or rebase, if configured)
git pull --rebase            # explicit rebase
git push                    # push current branch
git push -u origin feature  # set upstream and push
git push --force-with-lease  # safer than --force
```

`fetch` is non-destructive. `pull` modifies your branch. When something looks weird, `fetch` first and look.

Resolving conflicts

When a merge or rebase hits a conflict, Git pauses and marks the file:

```
<<<<<< HEAD
your version
=====
their version
>>>>>> feature-x
```

Edit the file, remove the markers, keep what's correct. Then:

```
git add file.txt            # mark as resolved
git merge --continue        # or: git rebase --continue
```

To bail out:

```
git merge --abort
git rebase --abort
```

For complex conflicts, a mergetool helps:

```
git mergetool              # opens your configured mergetool
```

Undoing changes

The right command depends on where the change is.

Goal	Command
Discard unstaged changes in a file	<code>git restore file.txt</code>
Unstage a file (keep working changes)	<code>git restore --staged file.txt</code>
Discard everything unstaged	<code>git restore .</code>
Undo last commit, keep changes staged	<code>git reset --soft HEAD~1</code>
Undo last commit, keep changes unstaged	<code>git reset HEAD~1</code> (default <code>--mixed</code>)
Undo last commit, throw away changes	<code>git reset --hard HEAD~1</code>
Make a new commit that undoes another	<code>git revert <commit></code>
Fix a typo in the last commit message	<code>git commit --amend</code>
Add forgotten file to last commit	<code>git add file && git commit --amend --no-edit</code>

`reset` rewrites history. `revert` adds a new commit that reverses an old one. Use `revert` on shared branches.

Stash

Save work-in-progress without committing.

```
git stash           # save uncommitted changes
git stash push -m "msg" # with a message
git stash list      # see all stashes
git stash show -p    # show diff of latest stash
git stash pop        # apply latest and remove
git stash apply      # apply but keep on the stash
git stash drop        # remove a stash
git stash clear      # remove all
```

Useful when you need to quickly switch branches but aren't ready to commit.

Stash skips untracked files by default. Use `git stash -u` to include them.

Tags

Annotated tags carry author, date, message, and a GPG signature if you sign them. Use them for releases.

```
git tag v1.0.0                # lightweight tag
git tag -a v1.0.0 -m "release 1.0.0" # annotated tag (preferred for releases)
git tag                        # list tags
git tag -l "v1.*"             # list matching
git push origin v1.0.0        # push a single tag
git push --tags               # push all tags
git tag -d v1.0.0             # delete local tag
git push --delete origin v1.0.0 # delete remote tag
```

.gitignore

Tells Git to ignore certain files. Lives at the repo root.

```
# Build output
target/
build/
dist/
*.class
*.jar

# IDE
.idea/
.vscode/
*.iml

# OS
.DS_Store
Thumbs.db

# Secrets
.env
*.key
*.pem

# Logs
*.log
logs/
```

Important: `.gitignore` only stops untracked files from being added. If a file is already tracked, you need:

```
git rm --cached file.txt
```

Then commit. The file stays on disk. Git stops tracking it.

Commit messages

The convention (Tim Pope's, adopted everywhere):

```
short summary (50 chars or less)
```

```
Longer body wrapped at 72 chars. Explain what changed and why.  
The diff already shows how. Reference issues or tickets.
```

```
Closes #123
```

Common formats:

- Conventional Commits: `feat:`, `fix:`, `chore:`, `docs:`, `refactor:`, `test:`. Plays well with automated changelogs.
- Plain prose: `Fix null pointer in order service when cart is empty.`

Whichever style, write in the imperative (“Add”, “Fix”, “Update”) so the message reads as completing the sentence “If applied, this commit will...”.

Bad: `fixed stuff`, `WIP`, `changes`, `asdf`.

Good: `Fix race condition in OrderService.placeOrder`.

Branching strategies

Trunk-based development

Everyone commits to `main` (or short-lived feature branches that merge fast). Continuous integration tests every push. Feature flags hide unfinished work.

Best for high-velocity teams shipping multiple times per day.

GitHub Flow

Branch off `main`, open a PR, merge back. No long-lived branches except `main`. Simple, popular for SaaS products.

Git Flow

Long-lived `main` and `develop` branches, plus `feature/*`, `release/*`, and `hotfix/*` branches. More structure, more overhead. Suited to versioned releases (libraries, on-prem products).

For most modern web teams, GitHub Flow or trunk-based wins. Git Flow's complexity costs more than it gives unless you're shipping numbered releases.

Advanced commands

cherry-pick

Apply a single commit from one branch onto another.

```
git checkout main
git cherry-pick abc1234
```

Useful for hotfixes: take a fix from a feature branch and apply it to a release branch.

bisect

Binary search through commit history to find the one that introduced a bug.

```
git bisect start
git bisect bad           # current commit is broken
git bisect good v1.0.0   # this old commit worked
# Git checks out a commit halfway between. Test it.
git bisect good         # or: git bisect bad
# Repeat. Git narrows down.
git bisect reset        # back to normal
```

Saves hours when you're hunting a regression in a long history.

blame

Who last changed each line of a file?

```
git blame file.txt
git blame -L 10,20 file.txt  # only lines 10-20
```

Useful for “who do I ask about this?” Often reveals the answer is “you, six months ago”.

reflog

Git's safety net. Every operation that moves HEAD is logged.

```
git reflog
```

If you ran `reset --hard` and lost work, `git reflog` shows the commit hash. Recover with:

```
git checkout abc1234      # detached HEAD
git branch recover abc1234 # save as a branch
```

Reflog entries expire (default 90 days). Treat it as a short-term safety net.

Hooks

Scripts that run on Git events. Live in `.git/hooks/`.

Common hooks:

- `pre-commit` : run linters, formatters, tests.
- `commit-msg` : validate commit message format.
- `pre-push` : run faster tests before push.
- `post-merge` : refresh local state after pull.

Hooks are local by default. Tools like `pre-commit` (Python) or Husky (Node.js) make them shareable across the team.

PR / MR workflow (GitHub, GitLab, Bitbucket)

The de facto workflow for code review:

1. Branch off `main`.
2. Make commits.
3. Push the branch to the remote.
4. Open a PR/MR (pull request / merge request).
5. CI runs (tests, lint, build).
6. Reviewers comment. You push more commits.
7. Once approved, merge (squash, merge commit, or rebase depending on team policy).
8. Delete the branch.

Merge strategies on PR:

- Merge commit: preserves all commits on the branch plus a merge commit. Most history.
- Squash and merge: combine all branch commits into one. Cleanest `main` history.
- Rebase and merge: replay branch commits on `main` with no merge commit. Linear history.

Pick a team-wide default. Mixing leads to chaos.

Best practices

- Commit often, push often. Frequent small commits beat rare big ones.
- Write meaningful commit messages. “Fix bug” tells the reader nothing.
- Branch per feature or fix. Don’t pile multiple ideas onto one branch.
- Pull before push. Save yourself the conflict on push.
- Prefer `--force-with-lease` over `--force`. Lease checks that the remote hasn’t moved since you fetched. Stops you from clobbering a teammate.
- Don’t commit secrets, large binaries, or generated files. Once in history, they’re hard to remove.
- Tag releases. Future you will need to compare versions.

- Configure a global `.gitignore` for OS and editor files. Keeps project `.gitignore` clean.
- Use SSH or HTTPS with a credential helper. Stop typing passwords.
- Sign your commits. GPG or SSH signing proves you authored them.

Common gotchas

- `git reset --hard` deletes uncommitted changes. Stash first if you might want them.
- `git pull` rebases or merges depending on config. Surprises new users. Set `pull.rebase` explicitly.
- Merging onto the wrong branch. Check `git status` before `git merge`. The current branch receives the merge.
- Force-pushing to shared branches. Rewrites history for everyone. Use `--force-with-lease` and team policy.
- Committed a secret. `git rm` doesn't help. It's still in history. Use BFG Repo-Cleaner or `git filter-repo`. Rotate the secret immediately.
- `.gitignore` doesn't ignore already-tracked files. Run `git rm --cached` on them first.
- Detached HEAD. Checking out a commit (not a branch) puts you in detached HEAD. Commits here aren't on any branch and can be lost.
- Stash includes nothing untracked by default. Use `git stash -u`.
- Line endings on Windows vs Unix. Configure `core.autocrlf` consistently across the team or use `.gitattributes`.
- Large repos slow Git down. Git struggles past a few GB. Use Git LFS for binaries.
- Tags and branches with the same name. Git complains. Rename one.
- `git rebase` on a public branch. Rewrites commit hashes. Anyone who pulled before will have a divergent history.

Quick reference

Operation	Command
Start a repo	<code>git init</code>
Clone a repo	<code>git clone <url></code>
See changes	<code>git status</code> , <code>git diff</code>
Stage a file	<code>git add <file></code>
Commit	<code>git commit -m "message"</code>
View history	<code>git log --oneline --graph</code>
Create branch	<code>git switch -c <branch></code>
Switch branch	<code>git switch <branch></code>
Merge branch	<code>git merge <branch></code>
Rebase	<code>git rebase <base></code>
Interactive rebase	<code>git rebase -i HEAD~N</code>
Push branch	<code>git push -u origin <branch></code>
Pull changes	<code>git pull</code>
Fetch only	<code>git fetch</code>
Undo last commit (keep changes)	<code>git reset HEAD~1</code>
Undo last commit (drop changes)	<code>git reset --hard HEAD~1</code>
Revert a commit	<code>git revert <hash></code>
Stash changes	<code>git stash</code> , <code>git stash pop</code>
Pick a commit from another branch	<code>git cherry-pick <hash></code>
Find a bad commit	<code>git bisect</code>
Recover lost work	<code>git reflog</code>